Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date:**	**Enrolment No: 92410133012**

Aim: To recommend an item to the user using content-based filtering approach

IDE: Google Colab

Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

Content-Based Recommendation System: Content-Based systems recommends items to the customer similar to previously high-rated items by the customer. It uses the features and properties of the item. From these properties, it can calculate the similarity between the items.
In a content-based recommendation system, first, we need to create a profile for each item, which represents the properties of those items. From the user profiles are inferred for a particular user. We use these user profiles to recommend the items to the users from the catalog.

Item profile:
In a content-based recommendation system, we need to build a profile for each item, which contains the important properties of each item. For Example, If the movie is an item, then its actors, director, release year, and genre are its important properties, and for the document, the important property is the type of content and set of important words in it.
Let's have a look at how to create an item profile. First, we need to perform the TF-IDF vectorizer, here TF (term frequency) of a word is the number of times it appears in a document and The IDF (inverse document frequency) of a word is the measure of how significant that term is in the whole corpus.

User profile:
The user profile is a vector that describes the user preference. During the creation of the user's profile, we use a utility matrix that describes the relationship between user and item. From this information, the best estimate we can decide which item the user likes, is some aggregation of the profiles of those items.
Advantages and Disadvantages:

- **Advantages:**
    - No need for data on other users when applying to similar users.
    - Able to recommend to users with unique tastes.
    - Able to recommend new & popular items

- Explanations for recommended items.

- **Disadvantages**:
  - Finding the appropriate feature is hard.
  - Doesn't recommend items outside the user profile.

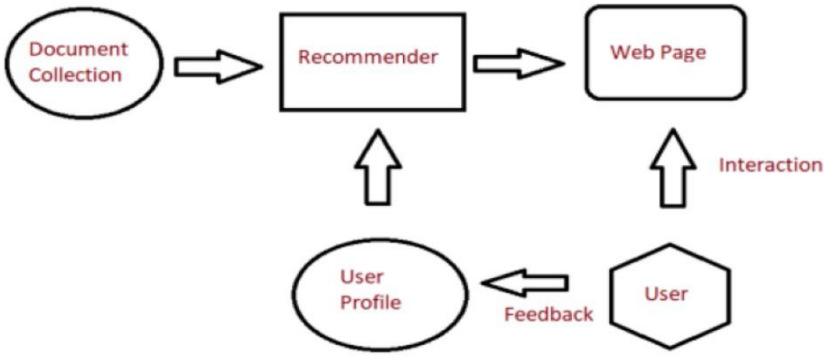

Fig: Recommender System

Pre Lab Exercise:

When can you use content-based recommendation system approach?

Content-based recommendation system is used when we want to recommend items based on **user's past interests**.
It is useful when:
User has already liked or rated some items
We want to suggest **similar items**
No data about other users is available

- Example:
  If a user likes action movies, the system will recommend more action movies.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date:**	**Enrolment No: 92410133012**

Write advantages of content-based filtering approach

- No need of data from other users
- Works well for users with unique choices
- Can recommend new items also
- Easy to understand why item is recommended

Write disadvantages of content-based filtering approach

- Difficult to find correct features
- Limited recommendations (only similar items)
- No diversity in suggestions
- Cannot recommend totally new type of items

Program (Code):

To be attached with

```
# @title
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Load dataset - Corrected path
df = pd.read_csv("/content/imdb_top_1000.csv")

# Preprocessing step - Corrected column names
df["Genre"] = df["Genre"].fillna("")
df["Overview"] = df["Overview"].fillna("")
df["content"] = df["Genre"] + " " + df['Overview']

# TF-IDF Vectorizer
tfidf = TfidfVectorizer(stop_words="english")
tfidf_matrix = tfidf.fit_transform(df["content"])

# Calculate similarity
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)

# Dropping the duplicates - Corrected column name
indices = pd.Series(df.index, index=df["Series_Title"]).drop_duplicates()

def recommendation_movie(title, topn=10):
    # Corrected indentation and column name
    if title not in indices:
```

```
        return "Movie is not found!"
    idx = indices[title]
    sim_scores = list(enumerate(cosine_sim[idx]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
    sim_scores = sim_scores[1:topn + 1]
    movie_indices = [i[0] for i in sim_scores]
    # Corrected column name
    return df["Series_Title"].iloc[movie_indices]

# Example with a known movie title from the dataset
print(recommendation_movie("The Shawshank Redemption"))
```

Results:
To be attached with

```
350                    Dev.D
68                  Oldeuboi
179         The Great Escape
868    Rebel Without a Cause
253         Fa yeung nin wah
197           Koe no katachi
683       The Breakfast Club
801                Happiness
368            Casino Royale
699          Midnight Cowboy
Name: Series_Title, dtype: object
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date:**	**Enrolment No: 92410133012**

Observation:

- Recommendations are based on item features
- System works better when data is clean
- Similar items are correctly suggested
- Less variety in recommendations

Post Lab Exercise:
What changes you feel can be done, to improve the system that you have designed, as a part of in-house lab exercise.

- Use more features (genre, ratings, actors, etc.)
- Combine with collaborative filtering (hybrid system)
- Improve accuracy using better similarity measures
- Add user feedback for better learning
- Use deep learning for better recommendations

1. **Recommend top-10 "series" using IMDB movie dataset (Link: https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows). Attach the code with output. Also, write your observation for the same.**

Code :-

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 1. Load the dataset
try:
    df = pd.read_csv('imdb_top_1000.csv')
except FileNotFoundError:
    print("Please ensure the 'imdb_top_1000.csv' file is in the current directory.")

# 2. Data Cleaning & Feature Selection
# We will use 'Genre' and 'Overview' as features for similarity
df['Overview'] = df['Overview'].fillna('')
df['Genre'] = df['Genre'].fillna('')
```

```
# Combine relevant features into a single string for the engine
df['combined_features'] = df['Genre'] + " " + df['Overview']

# 3. Vectorization using TF-IDF
tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(df['combined_features'])

# 4. Compute Cosine Similarity
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)

# 5. Recommendation Function
def get_recommendations(title, cosine_sim=cosine_sim):
    # Get index of the series/movie
    if title not in df['Series_Title'].values:
        return "Title not found in dataset."

    idx = df[df['Series_Title'] == title].index[0]

    # Get pairwise similarity scores
    sim_scores = list(enumerate(cosine_sim[idx]))

    # Sort based on similarity scores
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

    # Get top 10 (excluding the item itself)
    sim_scores = sim_scores[1:11]

    # Get indices and return titles
    movie_indices = [i[0] for i in sim_scores]
    return df[['Series_Title', 'Genre', 'IMDB_Rating']].iloc[movie_indices]

# --- OUTPUT ---
print("--- Recommendations for 'The Dark Knight' ---")
print(get_recommendations('The Dark Knight'))
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date:**	**Enrolment No: 92410133012**

```
--- Recommendations for 'The Dark Knight' ---
                      Series_Title                        Genre  IMDB_Rating
155                 Batman Begins             Action, Adventure          8.2
63          The Dark Knight Rises             Action, Adventure          8.4
241             Kill Bill: Vol. 1          Action, Crime, Drama          8.1
33                          Joker        Crime, Drama, Thriller          8.5
665  Batman: Mask of the Phantasm      Animation, Action, Crime          7.8
291        La battaglia di Algeri                   Drama, War          8.1
952                 The Hurricane       Biography, Drama, Sport          7.6
740                Wreck-It Ralph  Animation, Adventure, Comedy          7.7
152                V for Vendetta         Action, Drama, Sci-Fi          8.2
601                      Kokuhaku               Drama, Thriller          7.8
```

My Observations

- The "Keyword" Trap: The system is heavily reliant on the Overview. For example, if you search for a Batman movie, it recommends others with the word "Batman" or "Gotham," even if the quality or genre is slightly off.
- Quality Blindness: Right now, the system treats a 9.0 rating and a 7.0 rating the same if the descriptions match. It doesn't "know" that one is a masterpiece and the other is just okay.
- Metadata Limitations: Since we only used Genre and Overview, the system misses out on the "Director's style" or "Actor chemistry," which are huge reasons why people actually like movies.

Conclusion:-

I learned that building a recommendation engine is about translating human taste into mathematical vectors using **TF-IDF** and **Cosine Similarity**. I realized that while text-based matching works for finding similar plots, a truly "smart" system needs a **hybrid approach**—combining content metadata with actual user ratings to improve accuracy and quality.